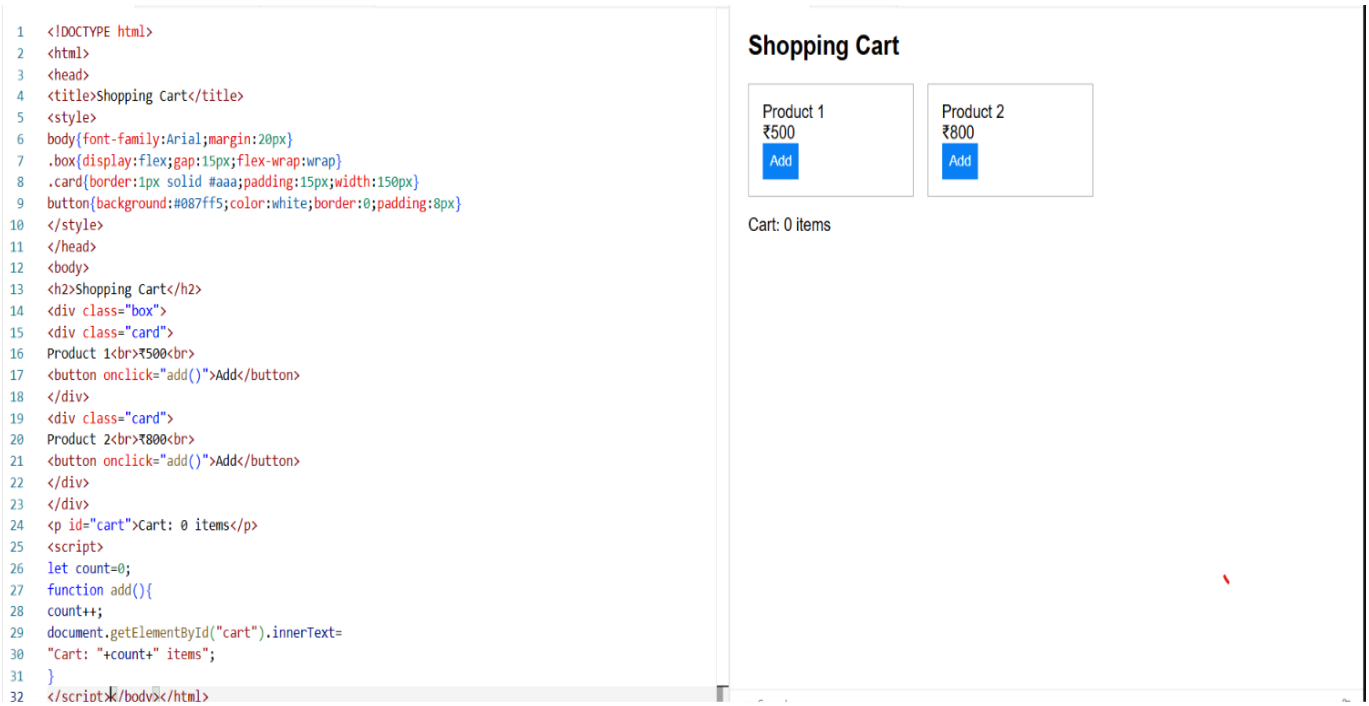
1. Responsive Shopping Cart – HTML, CSS Flex/Grid

Program: Save as an .html file and open it in a browser (unless noted otherwise).

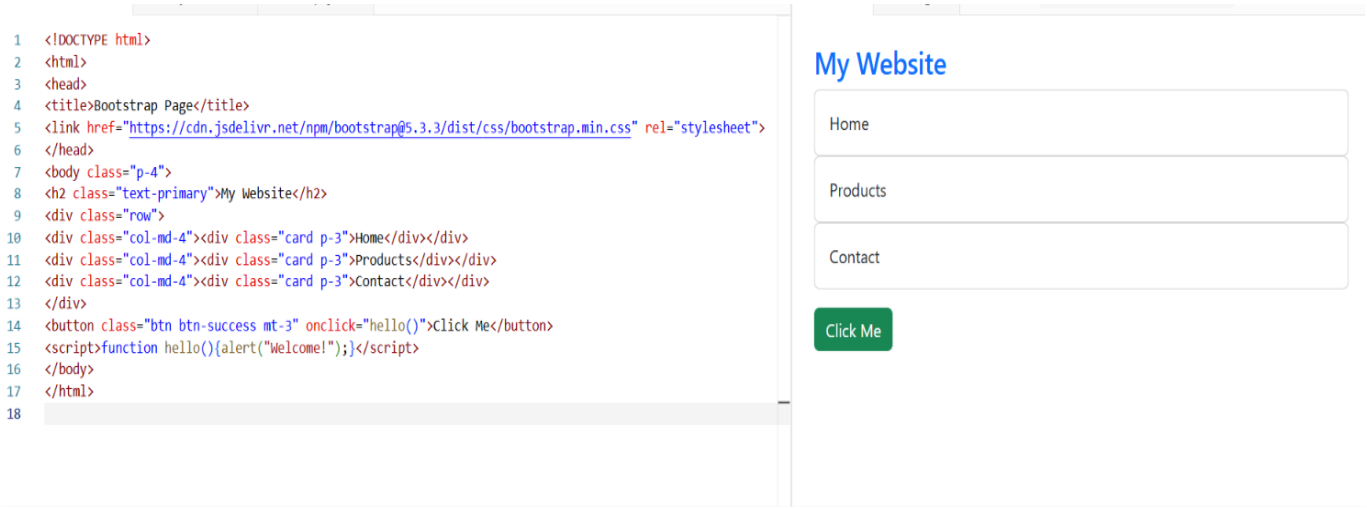
```
<!DOCTYPE html>
<html>
<head>
<title>Shopping Cart</title>
<style>
body{font-family:Arial;margin:20px}.box{display:flex;gap:15px;flex-wrap:wrap}
.card{border:1px solid #aaa;padding:15px;width:150px}
button{background:#087ff5;color:white;border:0;padding:8px}
</style>
</head>
<body>
<h2>Shopping Cart</h2>
<div class="box">
<div class="card">Product 1<br>₹500<br><button onclick="add('Product 1')">Add</button></div>
<div class="card">Product 2<br>₹800<br><button onclick="add('Product 2')">Add</button></div>
</div>
<p id="cart">Cart: 0 items</p>
<script>
let count=0;
function add(name){count++;document.getElementById("cart").innerText="Cart: "+count+" items";}
</script>
</body>
</html>
```



2. Responsive Web Page using Bootstrap

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head>
<title>Bootstrap Page</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="p-4">
<h2 class="text-primary">My Website</h2>
<div class="row">
<div class="col-md-4"><div class="card p-3">Home</div></div>
<div class="col-md-4"><div class="card p-3">Products</div></div>
<div class="col-md-4"><div class="card p-3">Contact</div></div>
</div>
<button class="btn btn-success mt-3" onclick="hello()">Click Me</button>
<script>function hello(){alert("Welcome!");}</script>
</body>
</html>
```

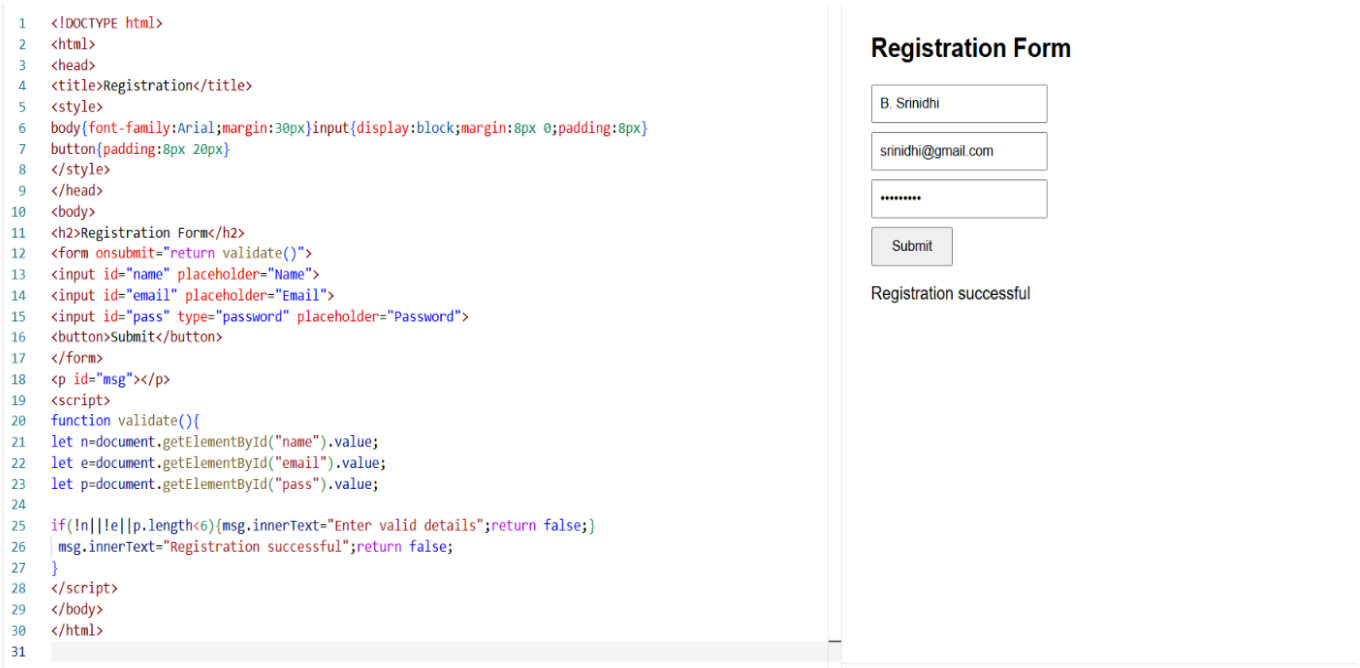


3. Client-Side Form Validation

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head>
<title>Registration</title>
<style>
body{font-family:Arial;margin:30px}input{display:block;margin:8px 0;padding:8px}
button{padding:8px 20px}
</style>
</head>
<body>
<h2>Registration Form</h2>
<form onsubmit="return validate()">
<input id="name" placeholder="Name">
<input id="email" placeholder="Email">
<input id="pass" type="password" placeholder="Password">
<button>Submit</button>
</form>
<p id="msg"></p>
<script>
function validate(){
let n=document.getElementById("name").value;
let e=document.getElementById("email").value;
let p=document.getElementById("pass").value;

if(!n||!e||p.length<6){msg.innerText="Enter valid details";return false;}
 msg.innerText="Registration successful";return false;
}
</script>
</body>
</html>
```



4. Weather Information with JavaScript (API Structure)

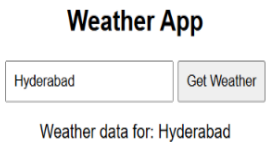
Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Weather</title>
<style>body{font-family:Arial;text-align:center;margin:40px}input,button{padding:8px}</style>
</head>
<body>
<h2>Weather App</h2>
<input id="city" placeholder="Enter city">
<button onclick="weather()">Get Weather</button>
<p id="out"></p>
<script>
async function weather() {
    let c=document.getElementById("city").value;
    let out=document.getElementById("out");

    if (!c){
        out.innerText="Enter a city";
        return;
    }

    out.innerText="Weather data for: "+c;
} // fetch("https://api.openweathermap.org/data/2.5/weather?q="+c+"&appid=API_KEY")
}
</script>
</body>
</html>
```





5. CRUD Operation – Simple Browser Demo

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>CRUD</title>
<style>body{font-family:Arial;margin:30px}input,button{padding:7px;margin:3px}</style>
</head>
<body>
<h2>Student CRUD</h2>
<input id="name" placeholder="Student name">
<button onclick="add()">Add</button>
<ul id="list"></ul>
<script>
let students=[];
function show(){
document.getElementById("list").innerHTML=students.map((x,i)=> `<li>${x} <button
onclick="edit(${i})">Edit</button>
 <button onclick="del(${i})">Delete</button></li>`).join("");
}
function add(){
let name=document.getElementById("name");
if(name.value){students.push(name.value);name.value="";show();}
}function edit(i){let x=prompt("New name",students[i]);if(x){students[i]=x;show();}}
function del(i){students.splice(i,1);show();}
</script>
</body>
</html>
```

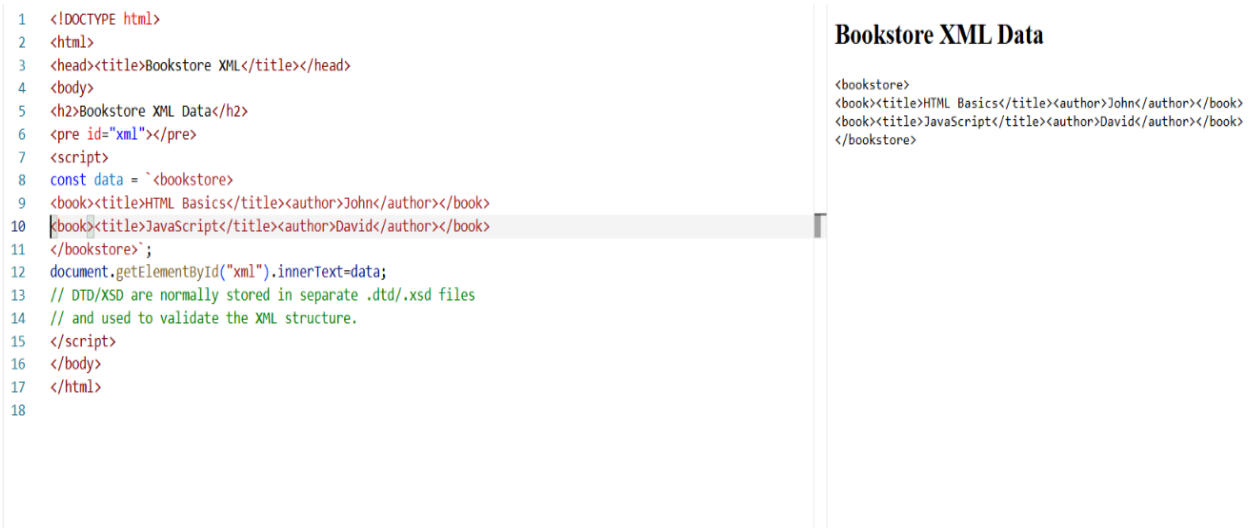




6. Bookstore XML – DTD/XSD Concept Demo

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Bookstore XML</title></head>
<body>
<h2>Bookstore XML Data</h2>
<pre id="xml"></pre>
<script>
const data = `<bookstore>
<book><title>HTML Basics</title><author>John</author></book>
<book><title>JavaScript</title><author>David</author></book>
</bookstore>`;
document.getElementById("xml").innerText=data;
// DTD/XSD are normally stored in separate .dtd/.xsd files
// and used to validate the XML structure.
</script>
</body>
</html>
```

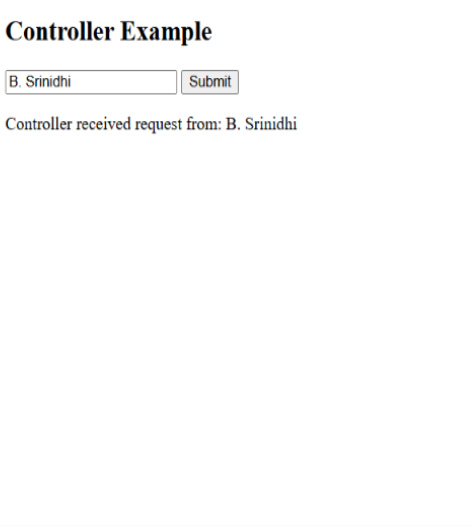


7. Servlet Controller – Simple Request/Response Demo

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Controller Demo</title></head>
<body>
<h2>Controller Example</h2>
<input id="user" placeholder="Enter name">
<button onclick="send()">Submit</button>
<p id="result"></p>
<script>
function send(){
 let u=document.getElementById("user").value;
 document.getElementById("result").innerText =
 "Controller received request from: "+u;
}
// In Java, a Servlet controller would receive the HTTP request,
// process it and send a response.
</script>
</body>
</html>
```

```
1  <!DOCTYPE html>
2  <html>
3  <head><title>Controller Demo</title></head>
4  <body>
5  <h2>Controller Example</h2>
6  <input id="user" placeholder="Enter name">
7  <button onclick="send()">Submit</button>
8  <p id="result"></p>
9  <script>
10 function send(){
11  let u=document.getElementById("user").value;
12  document.getElementById("result").innerText =
13  "Controller received request from: "+u;
14 }
15 // In Java, a Servlet controller would receive the HTTP request,
16 // process it and send a response.
17 </script>
18 </body>
19 </html>
20
```



8. Session Tracking using localStorage

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Session Tracking</title></head>
<body>
<h2>Login Session</h2>
<input id="user" placeholder="Username">
<button onclick="login()">Login</button>
<button onclick="logout()">Logout</button>
<p id="msg"></p>
<script>
function login(){
 localStorage.setItem("user",user.value);
 show();
}
function logout(){
 localStorage.removeItem("user");
 show();
}
function show(){
 msg.innerText=localStorage.getItem("user")
 ? "Logged in as "+localStorage.getItem("user"):"Not logged in";
}
show();
</script>
</body>
</html>
```

```
1  <!DOCTYPE html>
2  <html>
3  <head><title>Session Tracking</title></head>
4  <body>
5  <h2>Login Session</h2>
6  <input id="user" placeholder="Username">
7  <button onclick="login()">Login</button>
8  <button onclick="logout()">Logout</button>
9  <p id="msg"></p>
10 <script>
11 function login(){
12  localStorage.setItem("user",user.value);
13  show();
14 }
15 function logout(){
16  localStorage.removeItem("user");
17  show();
18 }
19 function show(){
20  msg.innerText=localStorage.getItem("user")
21  ? "Logged in as "+localStorage.getItem("user"):"Not logged in";
22 }
23 show();
24 </script>
25 </body>
26 </html>
27
28
```

Login Session

srinidhi Login Logout

Logged in as srinidhi

9. Node.js HTTP Server – Basic Concept

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Node.js Server</title></head>
<body>
<h2>Node.js HTTP Server</h2>
<pre>
const http = require("http");

http.createServer((req,res)=>{
  res.writeHead(200,{"Content-Type":"text/html"});
  res.end("Hello from Node.js Server");
}).listen(3000);

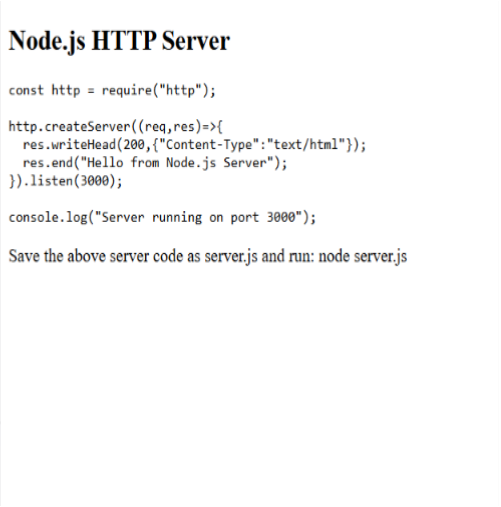
console.log("Server running on port 3000");
</pre>
<p>Save the above server code as server.js and run: node server.js</p>
</body>
</html>
```



```
<!DOCTYPE html>
<html>
<head><title>Node.js Server</title></head>
<body>
<h2>Node.js HTTP Server</h2>
<pre>
const http = require("http");

http.createServer((req,res)=>{
  res.writeHead(200,{"Content-Type":"text/html"});
  res.end("Hello from Node.js Server");
}).listen(3000);

console.log("Server running on port 3000");
</pre>
<p>Save the above server code as server.js and run: node server.js</p>
</body>
</html>
```



Node.js HTTP Server

```
const http = require("http");

http.createServer((req,res)=>{
  res.writeHead(200,{"Content-Type":"text/html"});
  res.end("Hello from Node.js Server");
}).listen(3000);

console.log("Server running on port 3000");
```

Save the above server code as server.js and run: node server.js

10. REST API CRUD – JavaScript Fetch Demo

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>REST API</title></head>
<body>
<h2>REST API Demo</h2>
<button onclick="load()">GET Users</button>
<pre id="out"></pre>
<script>
async function load(){
 const r=await fetch("https://jsonplaceholder.typicode.com/users");
 const data=await r.json();
 out.innerText=data.slice(0,3).map(x=>x.id+" - "+x.name).join("\n");
}
// POST, PUT and DELETE can be performed using fetch()
// with method:"POST", "PUT" or "DELETE".
</script>
</body>
</html>
```



11. JWT (JSON Web Token) – Simple Demonstration

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>JWT Demo</title></head>
<body>
<h2>JWT Login</h2>
<input id="user" placeholder="Username">
<button onclick="login()">Login</button>
<p id="msg"></p>
<script>
function login(){
 if(!user.value){msg.innerText="Enter username";return;}
 let token=btoa(JSON.stringify({user:user.value,exp:Date.now()+3600000}));
 localStorage.setItem("token",token);
 msg.innerText="Login successful. Token stored.";
}
// Real JWT applications create and verify signed tokens on a server.
</script>
</body>
</html>
```



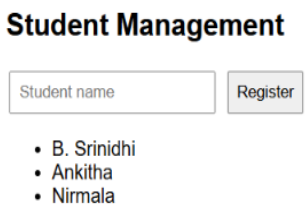
12. Student Management System

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head>
<title>Student Management</title>
<style>body{font-family:Arial;margin:30px}input,button{padding:7px;margin:3px}</style>
</head>
<body>
<h2>Student Management</h2>
<input id="student" placeholder="Student name">
<button onclick="add()">Register</button>
<ul id="list"></ul>
<script>
let a=[];
function add(){
 if(student.value){a.push(student.value);student.value="";show();}
}
function show(){list.innerHTML=a.map(x=>"<li>"+x+"</li>").join("");}
</script>
</body>
</html>
```



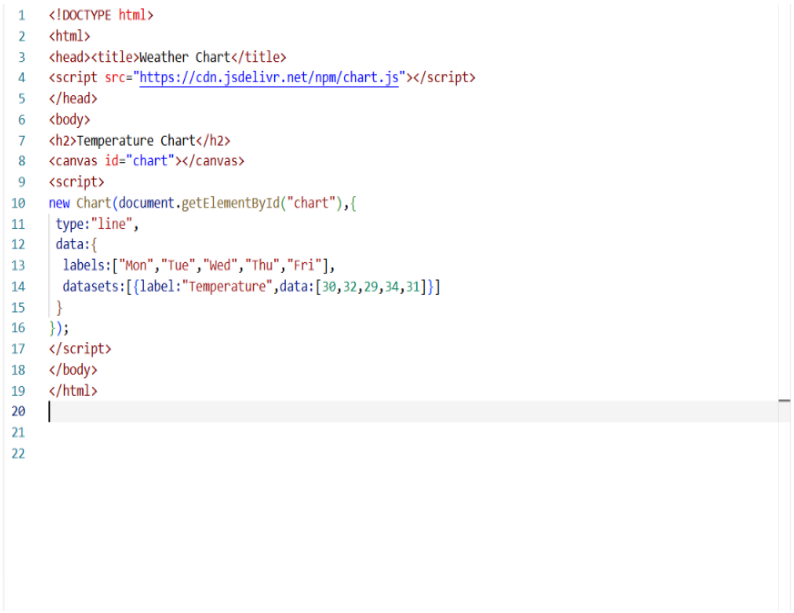
```
<!DOCTYPE html>
<html>
<head>
<title>Student Management</title>
<style>body{font-family:Arial;margin:30px}input,button{padding:7px;margin:3px}</style>
</head>
<body>
<h2>Student Management</h2>
<input id="student" placeholder="Student name">
<button onclick="add()">Register</button>
<ul id="list"></ul>
<script>
let a=[];
function add(){
 if(student.value){a.push(student.value);student.value="";show();}
}
function show(){list.innerHTML=a.map(x=>"<li>"+x+"</li>").join("");}
</script>
</body>
</html>
```

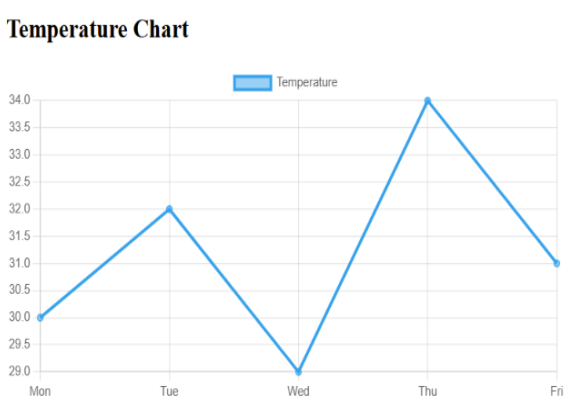


13. Weather Chart – Simple Graph

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head><title>Weather Chart</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h2>Temperature Chart</h2>
<canvas id="chart"></canvas>
<script>
new Chart(document.getElementById("chart"),{
 type:"line",
 data:{
  labels:["Mon","Tue","Wed","Thu","Fri"],
  datasets:[{label:"Temperature",data:[30,32,29,34,31]}]
 }
});
</script>
</body>
</html>
```





14. TODO Application – JavaScript Components Style

Program: Save as an .html file and open it in a browser (unless noted otherwise).

```
<!DOCTYPE html>
<html>
<head>
<title>TODO App</title>
<style>
body{font-family:Arial;margin:30px}li{margin:8px}
button{margin-left:10px}
</style>
</head>
<body>
<h2>TODO List</h2>
<input id="task" placeholder="Add task">
<button onclick="add()">Add</button>
<ul id="list"></ul>
<script>
let todos=[];
function add(){
 if(!task.value)return;
 todos.push(task.value);task.value="";show();
}
function remove(i){todos.splice(i,1);show();}
function show(){
 list.innerHTML=todos.map((x,i)=>
 `<li>${x}<button onclick="remove(${i})">X</button></li>`).join("");
}
</script>
</body>
</html>
```

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>TODO App</title>
5  <style>
6  body{font-family:Arial;margin:30px}li{margin:8px}
7  button{margin-left:10px}
8  </style>
9  </head>
10 <body>
11 <h2>TODO List</h2>
12 <input id="task" placeholder="Add task">
13 <button onclick="add()">Add</button>
14 <ul id="list"></ul>
15 <script>
16 let todos=[];
17 function add(){
18  if(!task.value)return;
19  todos.push(task.value);task.value="";show();
20 }
21 function remove(i){todos.splice(i,1);show();}
22 function show(){
23  list.innerHTML=todos.map((x,i)=>
24  `<li>${x}<button onclick="remove(${i})">X</button></li>`).join("");
25 }
26 </script>
27 </body>
28 </html>
29
```

